Combiners in MapReduce

Mahmoud Parsian

Ph.D. in Computer Science

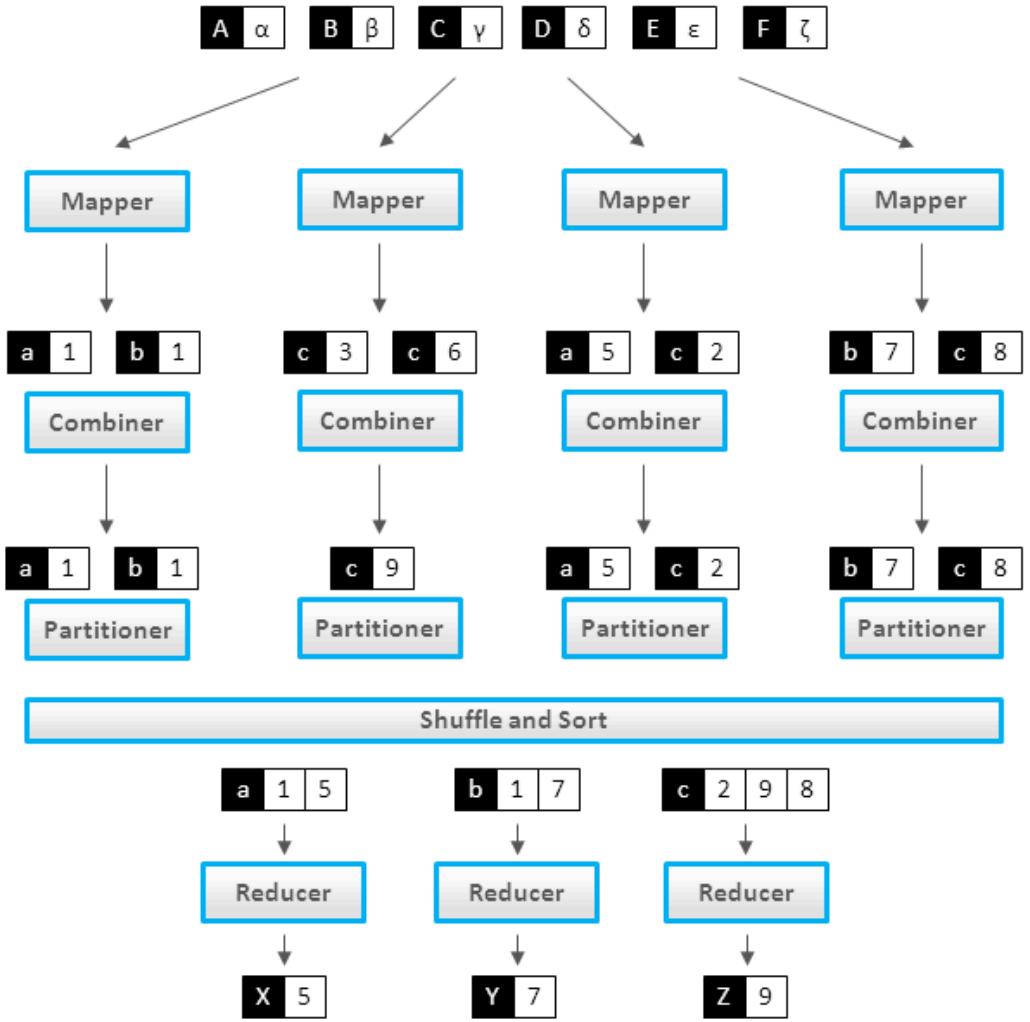
What Is a Combiner?

A **combiner** — also called a "mini-reducer" — summarizes a single mapper's output for one key *before* it ever leaves that mapper, i.e. before Sort & Shuffle ships it across the network.

```
map() -> combine() [OPTIONAL] -> partition/shuffle -> reduce()
```

It's optional, and it runs entirely locally, once per mapper — never across mappers.

Where It Sits in the Pipeline



The Correctness Requirement

A combiner is only safe to use if the reduce function is **associative and commutative** — because Spark/Hadoop are free to apply it zero, one, or many times, in any order, on any subset of a key's values, and the result still has to match running `reduce()` on the raw values directly.

```
Commutative:  F(a, b) = F(b, a)
Associative:  F(a, F(b, c)) = F(F(a, b), c)
```

`SUM` and `COUNT` satisfy both. **Not every aggregate does** — and getting this wrong produces a silently *wrong* answer, not a crash.

A Minimal Concrete Example

Two mappers, same key `K`, computing `SUM`:

```
Mapper 1's output for K: (K,2), (K,3), (K,4)
Mapper 2's output for K: (K,5), (K,6), (K,7), (K,8)
```

Without a combiner, all 7 raw values cross the network:

```
reduce(K, [2,3,4,5,6,7,8]) -> 35
```

With a combiner, each mapper pre-sums its own values *first* —

only 2 numbers cross the network instead of 7:

```
combine(K, [2,3,4]) -> 9          combine(K, [5,6,7,8]) -> 26
reduce(K, [9, 26]) -> 35           # same answer, less shuffled data
```

That's the Whole Idea

This is the whole trick, at the smallest possible scale — the fuller examples ahead just repeat it over more keys, more partitions, and (for `AVERAGE`) a value that needs `(sum, count)` instead of a bare number.

Where the Full Depth Already Lives

This repo already has a **1400+ line**, rigorously worked-out treatment of exactly this question — associativity, commutativity, the classic "average of an average is not an average" trap (and its fix), a catalog of which aggregates are safe (`SUM` , `MAX` , `COUNT`) and which aren't (`AVERAGE` , `MEDIAN` , subtraction, division), plus a design checklist and a partition-invariance test:

`associativity_and_commutativity/Associativity_Commutativity_and_Reducers.md`

Read that document for the *why* and the *how to check your own reducer*. Nothing in this deck said it better, so nothing here repeats it.

Worked Examples, With and Without a Combiner

Full side-by-side worked examples (same problem, solved once without a combiner and once with one, so you can see exactly what changes):

- `combiners/MapReduce_with_Combiners.md` — average, and `(avg, min, max)`, per gene
- `combiners/Word_Count_in_MapReduce.md` — Word Count, with a combiner

And in this folder specifically, the temperature-per-city example worked both ways:

- `08_mapreduce_example_without_combiners.md`
- `09_mapreduce_example_with_combiners.md`

References

1. Monoidify! Monoids as a Design Principle for Efficient MapReduce Algorithms — Jimmy Lin (and see Mahmoud's own `monoids/monoid_as_a_design_principle.md` covering the same ground)
2. *Data Algorithms* — Mahmoud Parsian
3. *Data Algorithms with Spark* — Mahmoud Parsian